

AI English Teacher P0

Creator: @Deepesh Yadav

Designer: Aadya

Design 

Problem Statement:

Currently, Riva is designed to be a conversation partner more than a teacher. Now we plan to experiment by converting Riva into an AI teacher that discovers the relevant topics for the user and teaches it via explanation, examples, and practice questions.

Overview

Riva onboards a paying learner, understands *why* they want to speak English, builds them a personalized sequence of topics to reach that goal, then teaches those topics one at a time using a per-topic lesson plan she grounds herself in — concluding each topic and rolling into the next, while always letting the learner steer.

The journey has three repeating "intelligences" (LLM calls) plus Riva-the-deliverer:

1. Intent Clarity Judge - makes sure we truly understand the learner's goal.
2. Curriculum Planner - turns (level + intent) into an ordered topic list.
3. Lesson-Plan Creator - turns (topic + level + intent) into a delivery plan.

Core Entities & Profile

This flow requires a persistent learner profile

Learner Profile

field	notes
name	name captured in greeting

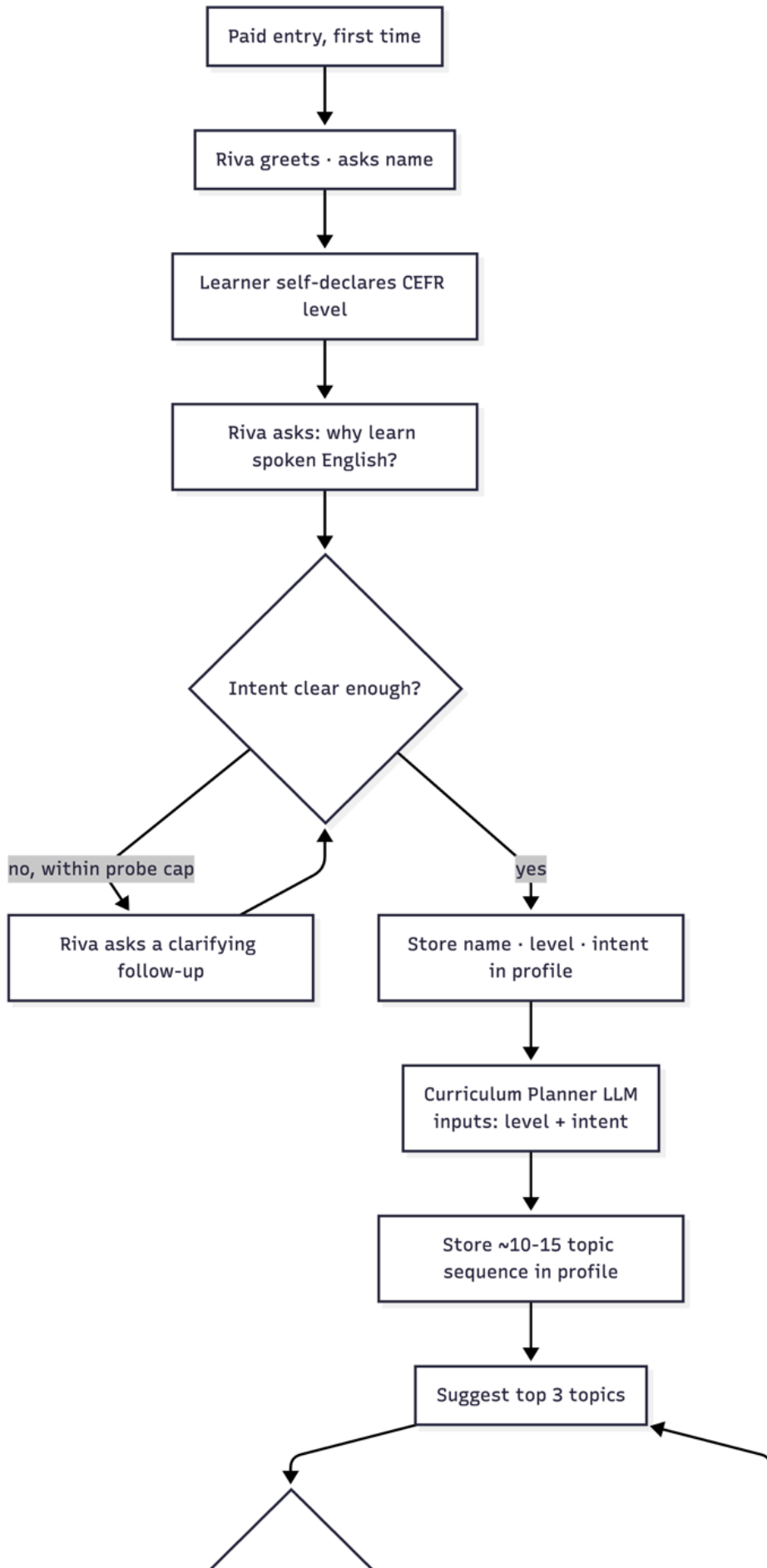
<code>self_declared_level</code>	CEFR A1–C2, chosen by the learner
<code>intent</code>	structured: { <code>raw_answer</code> , <code>summary</code> , <code>goal_contexts[]</code> , <code>motivation</code> , <code>clarity_status</code> }
<code>curriculum_plan</code>	ordered list of <code>Topic</code> s (the personalized sequence)
<code>lesson_plans</code>	per-topic delivery plans (created lazily when a topic is picked)
<code>progress</code>	current active topic + completed topics

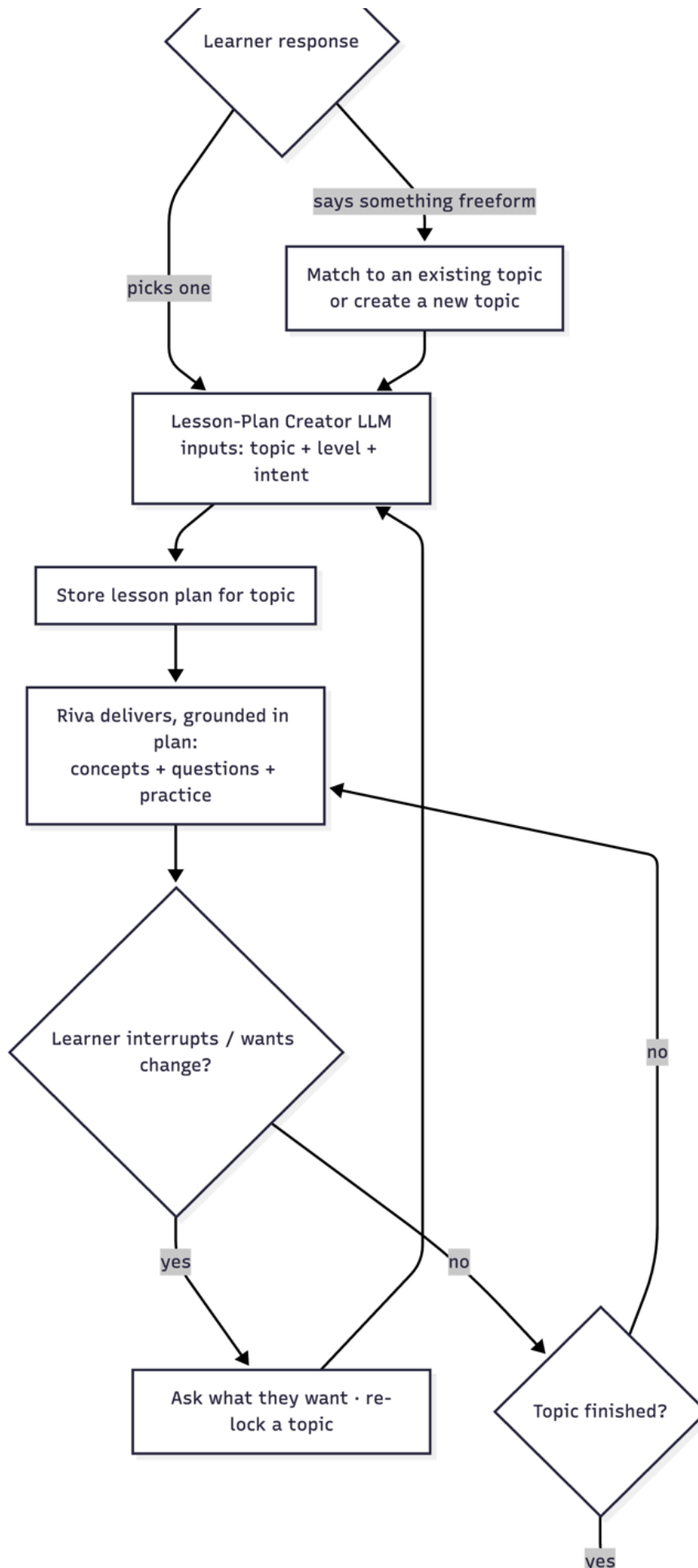
Topic (an item in the curriculum, or a newly created one)

field	notes
<code>id</code> , <code>title</code> , <code>description</code>	
<code>order</code>	position in the sequence
<code>status</code>	<code>pending</code> <code>active</code> <code>completed</code>

Lesson Plan (per topic)

field	notes
<code>topic_id</code>	
<code>steps[]</code>	ordered delivery steps, each typed: <code>concept</code> <code>question</code> <code>practice</code> <code>recap</code>







The Intelligences (LLM services)

1. Intent Clarity Judge

- **When:** after each learner answer to "why do you want to learn spoken English?"
- **Input:** the learner's reply (+ the short exchange so far).
- **Output:** `{ clear: bool, structured_intent?: {summary, goal_contexts[], motivation}, follow_up_question?: string }`.
- "Sufficiently clear" = we can name the situations/contexts they want English for (e.g., job interviews, work meetings, travel, daily conversation) with enough specificity to differentiate the curriculum, not just "to improve." Bounded by a probe cap (e.g., max 2 follow-ups) after which we proceed with best understanding.

2. Curriculum Planner

- **When:** once intent is captured.
- **Input:** `self_declared_level`, `structured_intent`.
- **Output:** an ordered list of ~10-15 topics, each `{ title, description, order }`, sequenced to progress toward the goal.
- **Stored as** `curriculum_plan`

3. Lesson-Plan Creator

- **When:** a topic is locked.
- **Input:** the `topic`, `self_declared_level`, `intent`.
- **Output:** a lesson plan = ordered, typed `steps[]` (`concept` / `question` / `practice` / `recap`), each with its content/prompt; collectively defining what "delivering this topic" means (and therefore its end).
- **Stored** per topic.

4. Riva (the deliverer)

- Grounds herself in the active lesson plan; delivers concepts, asks questions, runs practice; keeps the learner talking; actively and kindly corrects; detects topic completion and interruptions; hands back to the loop.

Delivery Model

Delivery is driven by the lesson plan's typed steps, surfaced to the UI on each Riva message:

Type	What it is	UI treatment
concept	An explanation Riva delivers	Normal chat bubble
question	A question the learner must respond to An invitation to speak / produce	Visually differentiated
recap	Topic conclusion summary	Normal chat bubble

Riva moves through the plan's steps in order but adapts (re-explains, gives another example, slows down) and always yields to learner interruptions.

UI Requirements

- **Differentiated question rendering:** when Riva's asks a `question`, the UI renders it **distinctly** from normal explanation (e.g., a highlighted question card), so the learner clearly recognizes "I'm being asked something."
 - question could be of two types
 - SAR type question
 - for SAR-type questions, the user will speak into the mic, and we will show the response in terms of what words he got correct and what words were incorrect
 - open-ended question
 - open-ended question will be answered like a normal user response, and then Riva will respond to it, whether there was some scope of improvement, or it will give feedback
- **Topic suggestion UI:** present the **top 3 topics** as selectable options

Edge Cases

Case	Handling
Intent stays vague after the probe cap	Proceed with best-effort intent; refine later as Riva learns more
Self-declared level seems wrong during delivery	Riva adapts difficulty in the moment
Freeform request matches an existing topic	Lock that topic; continue the sequence
Freeform request matches nothing	Create a new topic ; it's inserted into the topic sequence and counts toward the goal
Learner finishes all planned topics	Offer to extend the plan / regenerate next topics toward the goal
Learner interrupts repeatedly	Always honor it; re-lock quickly; avoid forcing them back